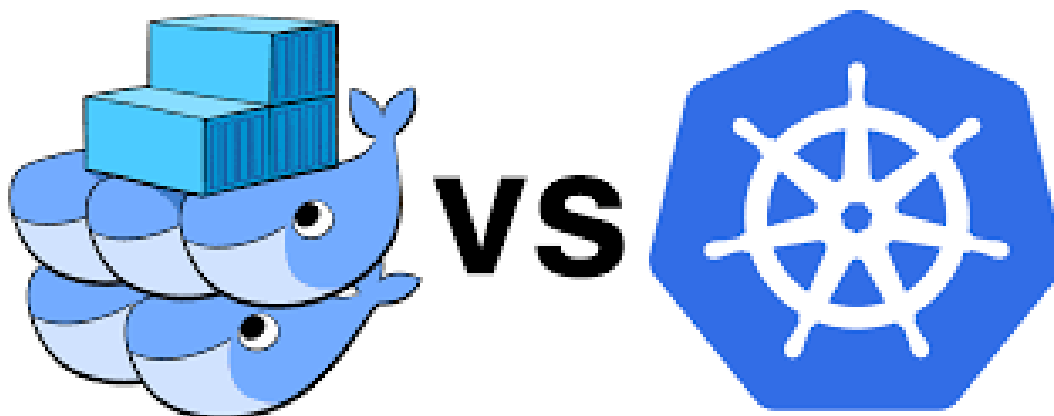


Orquestradors: Docker Swarm i Kubernetes



Índex

FASE 1: Desplegament de la infraestructura base, microserveis i seguretat SSL	4
1. Introducció i Objectiu del Projecte	4
2. Especificacions de l'Entorn d'Infraestructura	4
2.1. Plataforma de Virtualització i Nodes	4
3. Seguretat i Resolució de Noms	5
3.1. Implementació de Capa de Seguretat SSL (HTTPS)	5
3.2. Configuració de FQDN (DNS Local)	5
4. Lògica de Negoci i Interfície d'Usuari	6
4.1. Gestió d'Usuaris i Seguretat de Sessions	6
Procés de Registre i Seguretat de Dades	6
Control d'Accés i Gestió de Sessions	7
4.2. UX i Catàleg Dinàmic	8
5. El Cicle de Compra i Sincronització de Stock	8
6. Documentació Tècnica de l'Arquitectura de Fitxers	9
6.1. Configuració de la Infraestructura (Orquestració)	9
6.2. Lògica del Backend i Persistència (PHP/MySQL)	10
6.3. Recursos Estàtics i Multimèdia	13
Fase 2: Orquestració i Alta Disponibilitat amb Docker Swarm	14
7. Inicialització del clúster Swarm	14
8. Adaptació del fitxer a Docker Stack	15
8.1. Millores en el fitxer Compose i Gestió de Sessions	16
8.1.1. Gestió de Rèpliques i Nodes (Service Scaling)	16
8.1.2. Seguretat de Dades i Punt d'Entrada (Manager Constraints)	16
8.1.3. Comunicació entre Nodes (Xarxes Overlay)	16
8.2. Configuració del Proxy i Persistència (Nginx)	17
9. Desplegament de l'Stack	17
10. Prova de tolerància a fallades	18
11. Escalat en calent	19
Fase 3: Seguretat al Clúster Docker Swarm	19
12. Anàlisi de vulnerabilitats inicial	19
13. Gestió de credencials amb Docker Secrets	20
14. Aïllament de xarxes Overlay (Segmentació)	20
A. Configuració de la xarxa interna (YAML)	20
B. Prova d'aïllament real (La prova del "Ping")	20
15. TLS i Certificats al clúster	21
16. Escaneig d'imatges (Imatge Segura)	21
Fase 4: Kubernetes — Migració i Gestió Avançada	22

INS Sa Palomera

ASIX – Projecte Final de Curs 2025-2026

Grup: Izan Ruiz · Youssef Fouad · Adrià Rodríguez

17. Estudi i comparativa: Docker Swarm vs Kubernetes	22
18. Muntatge de l'entorn amb Minikube	22
19. Creació de fitxers YAML	23
19.1. Persistència amb PVC (k8s-pvc.yaml)	23
19.2. Base de Dades i Seguretat (k8s-db.yaml)	23
19.3. Microservei de Productes (k8s-product.yaml)	24
19.4. Gateway d'accés (k8s-gateway.yaml)	25
20. Creació del Namespace: shopmicro	25
21. Migració i desplegament	26
22. Rolling update automàtic	26
23. Comprovació dels fluxos funcionals	27

FASE 1: Desplegament de la infraestructura base, microserveis i seguretat SSL

1. Introducció i Objectiu del Projecte

L'objectiu principal d'aquest projecte és el disseny i desplegament d'una plataforma d'e-commerce especialitzada en maquinari de xarxa professional. A diferència d'una aplicació monolítica tradicional, s'ha optat per una arquitectura basada en microserveis gestionada mitjançant Docker Compose.

Aquest enfocament permet separar les responsabilitats (frontend, lògica de negoci i base de dades) en contenidors independents, facilitant el manteniment, l'escalabilitat i la seguretat del sistema global. S'ha posat especial èmfasi en la seguretat de les dades de l'usuari i en la integritat del stock mitjançant transaccions amb la base de dades.

2. Especificacions de l'Entorn d'Infraestructura

El projecte s'ha muntat sobre un entorn de virtualització controlat per garantir el rendiment dels contenidors i la correcta gestió de les interfícies de xarxa.

2.1. Plataforma de Virtualització i Nodes

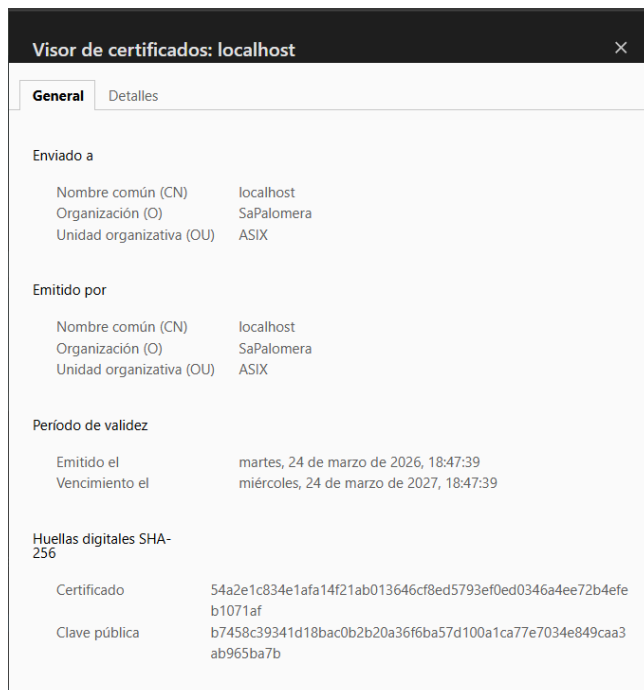
- **Sistema Host:** Ubuntu Desktop 22.04 LTS.
- **Motor de Contenidors:** Docker Engine v24.0.x i Docker Compose v2.x.
- **Segmentació de Xarxes (Docker Networks):**
 - **frontend-net (Bridge):** Aïlla el trànsit del Proxy Invers amb l'exterior.
 - **backend-net (Bridge):** Xarxa privada exclusiva per a la comunicació interna entre el servidor d'aplicacions i el motor de base de dades.

3. Seguretat i Resolució de Noms

3.1. Implementació de Capa de Seguretat SSL (HTTPS)

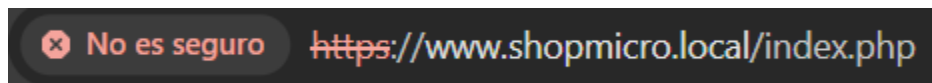
S'ha configurat un entorn de navegació segura mitjançant el protocol TLS/SSL:

- **Terminació SSL:** Nginx rep la petició xifrada (port 443) i la desxifra abans d'enviar-la al servei intern, optimitzant el rendiment.
- **Redirecció 301:** Qualsevol intent d'accés per HTTP (port 80) es redirigeix automàticament a HTTPS per evitar la transmissió de dades en text pla.



3.2. Configuració de FQDN (DNS Local)

S'ha implementat un sistema de resolució de noms mitjançant el fitxer de mapeig d'hosts de l'ordinador client. Això permet que l'aplicació respongui al domini **www.shopmicro.local**, validant la identitat del lloc amb el certificat SSL instal·lat.



4. Lògica de Negoci i Interfície d'Usuari

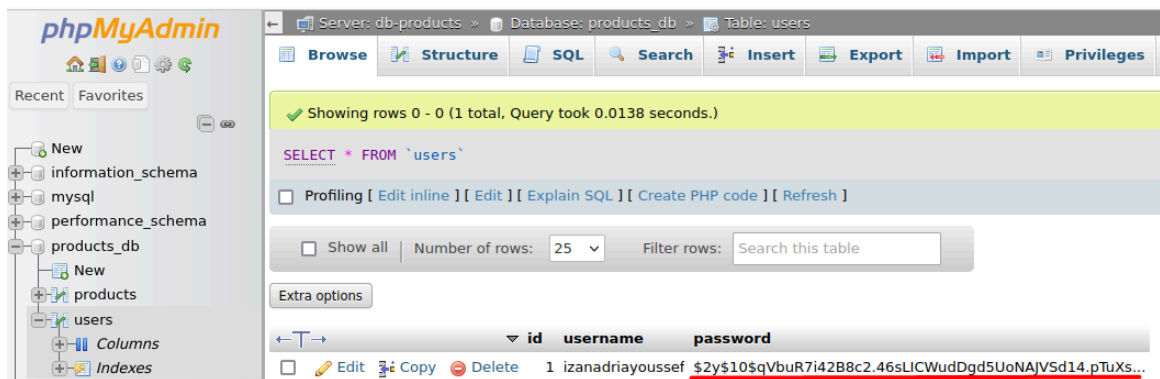
4.1. Gestió d'Usuaris i Seguretat de Sessions

La seguretat és un dels pilars d'aquest projecte. S'ha implementat un sistema complet de gestió d'usuaris que garanteix que les dades sensibles estiguin protegides i que només els usuaris autoritzats puguin realitzar compres o accedir al panell de control.

Procés de Registre i Seguretat de Dades

En lloc de guardar les dades de l'usuari de forma plana, s'ha aplicat una capa de seguretat seguint els estàndards actuals de la indústria:

- **Xifrat BCrypt:** Quan un usuari es registra, la seva contrasenya no es guarda tal qual a MySQL. S'utilitza la funció `password_hash()` de PHP amb l'algorisme BCrypt. *Què vol dir això?* Que si algú aconseguís accedir a la base de dades, només veuria una cadena de caràcters il·legible. És un procés d'un sol sentit: no es pot desxifrar la contrasenya original a partir del hash.



- **Integritat de les dades:** Durant el registre, el sistema valida que l'usuari no existeixi prèviament, evitant duplicitats i assegurant que cada compte estigui vinculat a una identitat única al sistema.

Control d'Accés i Gestió de Sessions

Perquè el servidor sàpiga qui és cada usuari mentre navega, s'ha implementat un sistema de **sessions de PHP**:

1. **Inici de Sessió (Login):** Quan l'usuari introdueix les seves credencials, el sistema recupera el hash de la base de dades i el compara amb la contrasenya introduïda mitjançant `password_verify()`. Si és correcte, es genera un ID de sessió únic al servidor.



The image shows a login form with a dark background. At the top, it says 'Iniciar Sessió'. Below that are two input fields: the first contains the username 'izanadriayoussef' and the second contains masked characters '....'. A blue button labeled 'Entrar' is below the password field. At the bottom, it says 'No tens compte? [Registrat](#)'.

2. **Persistència:** Mentre la sessió estigui oberta, l'usuari pot navegar per tota la botiga sense haver de tornar a posar la contrasenya a cada pàgina. El servidor "recorda" qui és.
3. **Tancament de Sessió Segur (Logout):** S'ha programat un fitxer específic, `logout.php`. Quan l'usuari decideix sortir, aquest fitxer no només el redirigeix a la pàgina principal, sinó que destrueix completament la sessió al servidor i neteja les galetes (cookies) del navegador. Això evita que algú pugui suplantar l'usuari si fa servir el mateix ordinador després.

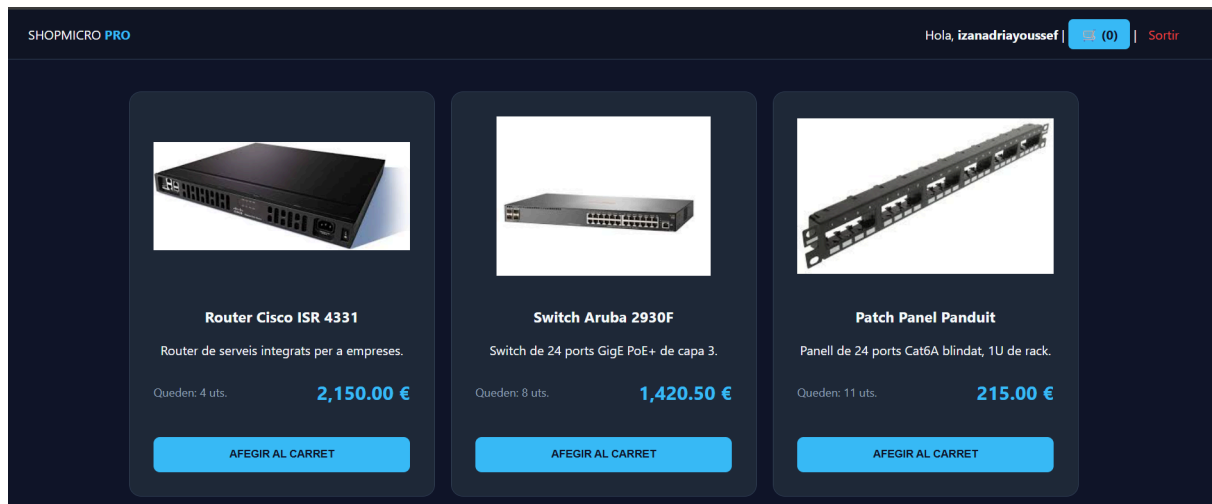


The image shows a user profile bar with a dark background. It starts with 'Hola, izanadriayoussef |'. To the right is a blue button with a notification icon and the text '(0)'. Further right is a red button labeled 'Sortir'.

4.2. UX i Catàleg Dinàmic

En aquesta fase, s'ha evolucionat l'aplicació des d'un prototip funcional cap a una plataforma professional, centrant-se en dos pilars: l'estètica visual i la gestió eficient de les dades.

- **Interfície Visual:** Cada producte (Router, Switch, Patch Panel.) es presenta en una "Card" amb la seva fotografia real i metadades detallades (preu, descripció i stock).
- **Abstracció del Contingut:** Les rutes de les imatges i els preus s'extreuen de MySQL en temps real, permetent escalar el catàleg sense modificar el codi font HTML/PHP.



5. El Cicle de Compra i Sincronització de Stock

La funcionalitat més complexa és la gestió automatitzada de l'inventari:

1. **Validació:** El sistema comprova en base de dades la disponibilitat abans de processar.
2. **Actualització Atòmica:** S'executa una sentència **UPDATE** a MySQL que resta les unitats venudes del camp **stock**.
3. **Feedback Visual:** L'usuari rep una confirmació neta ("¡Compra realizada!").
4. **Bloqueig de Stock:** Si el stock arriba a 0, el botó "Afegir" es desactiva automàticament mostrant el text **"Sense Stock"**.

6. Documentació Tècnica de l'Arquitectura de Fitxers

Aquesta secció detalla la configuració del sistema de fitxers que permet l'orquestració de la infraestructura.

6.1. Configuració de la Infraestructura (Orquestració)

- **docker-compose.yml:** Fitxer mestre en format YAML que defineix el desplegament. Estableix les imatges oficials, el mapeig de ports (80/443), la persistència de dades mitjançant volums i l'aïllament de xarxes bridge.

```
GNU nano 6.2                docker-compose.yml *
version: '3.8'

services:
  # Base de dades MySQL per a usuaris, productes i stock
  db-products:
    image: mysql:8.0
    container_name: db-products
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: products_db
    volumes:
      - db_products_data:/var/lib/mysql
    networks:
      - backend-net
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Microservei PHP amb Apache (Gestiona la web, login i carret)
  product-service:
    build: ./services/product-service
    container_name: product-service
    depends_on:
      db-products:
        condition: service_healthy
    networks:
      - backend-net

# API Gateway (Nginx) amb suport SSL/HTTPS
api-gateway:
  image: nginx:latest
  container_name: api-gateway
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    - ./services/nginx/ssl:/etc/nginx/ssl:ro
  networks:
    - frontend-net
    - backend-net
  depends_on:
    - product-service

# PHPMYADMIN
phpmyadmin:
  image: phpmyadmin:latest
  container_name: phpmyadmin-service
  ports:
    - "8080:80"
  environment:
    PMA_HOST: db-products
    MYSQL_ROOT_PASSWORD: root_password
  networks:
    - backend-net

# Definició de les xarxes aïllades
networks:
  frontend-net:
  backend-net:

# Volum persistent per a la base de dades
volumes:
  db_products_data:
```

- **nginx.conf:** Configuració del Proxy Invers que actua com a porta d'entrada segura. Gestiona el xifratge SSL, defineix els certificats de seguretat i redirigeix el trànsit extern cap al contenidor de l'aplicació.

```
GNU nano 6.2                nginx.conf
server {
    listen 80;
    server_name www.shopmicro.local;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name www.shopmicro.local;

    ssl_certificate /etc/nginx/ssl/shopmicro.crt;
    ssl_certificate_key /etc/nginx/ssl/shopmicro.key;

    location / {
        proxy_pass http://product-service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

6.2. Lògica del Backend i Persistència (PHP/MySQL)

- **db.php:** Control d'accés a dades. Inclou la configuració de la connexió (host, user, pass) i un script d'autoinstal·lació que genera l'esquema de taules relacional (users i products) en el primer arrencada.

```

GNU nano 6.2                                db.php
<?php
// services/product-service/app/db.php
session_start();
$conn = new mysqli('db-products', 'root', 'root_password', 'products_db');

if ($conn->connect_error) { die("Error DB"); }

// Taula de Productes actualitzada
$conn->query("CREATE TABLE IF NOT EXISTS products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100),
  descripcio TEXT,
  preu DECIMAL(10,2),
  stock INT,
  imatge VARCHAR(100)
)");

// Taula d'Usuaris
$conn->query("CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE,
  password VARCHAR(255)
)");

// Dades inicials amb els noms de les imatges reals
$check = $conn->query("SELECT id FROM products LIMIT 1");
if ($check->num_rows == 0) {
  $conn->query("INSERT INTO products (nom, descripcio, preu, stock, imatge) VALUES
    ('Router Cisco ISR 4331', 'Router de serveis integrats per a empreses.', 2150.00, 4, 'cisco.jpg'),
    ('Switch Aruba 2930F', 'Switch de 24 ports GigE PoE+ de capa 3.', 1420.50, 8, 'aruba.jpg'),
    ('Patch Panel Panduit', 'Panell de 24 ports Cat6A blindat, 1U de rack.', 215.00, 15, 'panduit.jpg')");
}
?>

```

- **register.php:** Mòdul d'alta d'usuaris. Captura les dades del formulari, aplica l'algorisme de xifratge BCrypt a la contrasenya i executa la sentència INSERT per registrar el nou perfil a la base de dades.

```

GNU nano 6.2                                register.php *
<?php include 'db.php';
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
  $user = $_POST['username'];
  // Encriptem la contrasenya per seguretat
  $pass = password_hash($_POST['password'], PASSWORD_DEFAULT);

  $stmt = $conn->prepare("INSERT INTO users (username, password) VALUES (?, ?)");
  $stmt->bind_param("ss", $user, $pass);

  if ($stmt->execute()) {
    header("Location: login.php");
  } else {
    $error = "El usuario ya existe o hay un error.";
  }
}

?>

```

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="css/style.css">
  <title>Registro - ShopMicro</title>
</head>
<body>
  <div class="container">
    <div class="card" style="max-width: 400px;">
      <h2>Crear Cuenta</h2>
      <form method="post">
        <input type="text" name="username" placeholder="Nombre de usuario" required class="input">
        <input type="password" name="password" placeholder="Contraseña" required class="input">
        <button type="submit" class="btn">Registrarse</button>
      </form>
      <p>¿Ya tienes cuenta? <a href="login.php">Inicia sesión aquí</a></p>
      <?php if(isset($error)) echo "<p style='color:red'>$error</p>"; ?>
    </div>
  </div>
</body>
</html>

```

- **login.php:** Sistema d'autenticació. Verifica les credencials introduïdes mitjançant `password_verify()`, inicia la sessió de l'usuari i genera les variables de control que permeten l'accés a les funcions de compra.

```

GNU nano 6.2 login.php
<?php include 'db.php';
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $user = $_POST['username'];
    $pass = $_POST['password'];

    $res = $conn->query("SELECT * FROM users WHERE username='$user'");
    $u = $res->fetch_assoc();

    if ($u && password_verify($pass, $u['password'])) {
        $_SESSION['user'] = $u['username'];
        header("Location: index.php");
    } else {
        $error = "Usuari o contrasenya incorrectes.";
    }
}

<?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <link rel="stylesheet" href="css/style.css">
    <title>Login - ShopMicro</title>
</head>
<body>
    <div class="container">
        <div class="card" style="max-width: 400px;">
            <h2>Iniciar Sessió</h2>
            <form method="post">
                <input type="text" name="username" placeholder="Usuario" required class="input">
                <input type="password" name="password" placeholder="Contraseña" required class="input">
                <button type="submit" class="btn">Entrar</button>
            </form>
            <p>No tens compte? <a href="register.php">Registrar</a></p>
            <?php if(isset($error)) echo "<p style='color:red'>$error</p>"; ?>
        </div>
    </div>
</body>
</html>

```

- **index.php:** Controlador principal de la vista. Realitza la consulta SQL de selecció i gestiona l'array de sessió `$_SESSION['cart']` per acumular productes.

```

GNU nano 6.2 index.php
<?php include 'db.php';
// Protección de ruta
if (!isset($_SESSION['user'])) { header("Location: login.php"); exit; }

if (isset($_POST['add_to_cart'])) {
    $id = $_POST['product_id'];
    $_SESSION['cart'][$id] = ($_SESSION['cart'][$id] ?? 0) + 1;
}

<?>

<!DOCTYPE html>
<html lang="ca">
<head>
    <meta charset="UTF-8">
    <title>ShopMicro | Botiga Online Pro</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
    <nav class="navbar">
        <div class="logo">SHOPMICRO <span>PRO</span></div>
        <div>
            Hola, <b><?php echo $_SESSION['user']; ?></b> |
            <a href="cart.php" class="cart-btn"><?php echo isset($_SESSION['cart']) ? array_sum($_SESSION['cart']) : 0; ?></a> |
            <a href="logout.php" style="color:#ef4444; margin-left:10px; text-decoration:none;">Sortir</a>
        </div>
    </nav>

    <main class="container">
        <?php
            $res = $conn->query("SELECT * FROM products");
            while($p = $res->fetch_assoc()):
                $dispo = $p['stock'] > 0;
                <div class="card <?php echo $dispo ? 'out' : '' ?>>
                    " class="product-image">
                    <h3><?php echo $p['nom']; ?></h3>
                    <p><?php echo $p['descripcio']; ?></p>
                    <div class="info-row">
                        <span class="stock">Queden: <?php echo $p['stock']; ?> uts.</span>
                        <span class="price"><?php echo number_format($p['preu'], 2); ?> €</span>
                    </div>
                    <form method="post">
                        <input type="hidden" name="product_id" value="<?php echo $p['id']; ?>">
                        <button type="submit" name="add_to_cart" class="btn" <?php echo $dispo ? 'disabled' : '' ?>>
                            <?php echo $dispo ? 'AFEGIR AL CARRET' : 'SENSE STOCK'; ?>
                        </button>
                    </form>
                </div>
            <?php endwhile; ?>
        </main>
    </body>
</html>

```

- **cart.php:** Mòdul transaccional. És l'encarregat de tancar la sessió de compra i executar les operacions DML (UPDATE) sobre la base de dades per actualitzar els valors d'inventari.

```
GNU nano 6.2                                cart.php
<?php include 'db.php';
// Protección de ruta
if (!isset($_SESSION['user'])) { header("Location: login.php"); exit; }

// Lògica del pagament (descompte de stock)
if (isset($_POST['buy']) && !empty($_SESSION['cart'])) {
    foreach ($_SESSION['cart'] as $id => $qnt) {
        $conn->query("UPDATE products SET stock = stock - $qnt WHERE id = $id AND stock >= $qnt");
    }
    $_SESSION['cart'] = []; // Buitem el carret
    $success = true; // Variable per mostrar el missatge net
}
?>

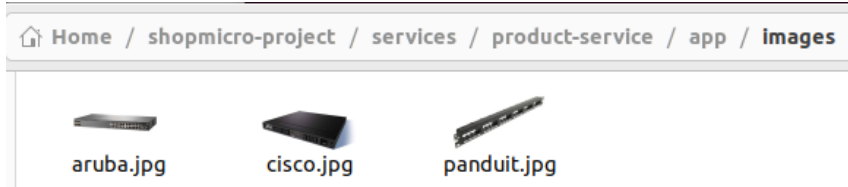
<!DOCTYPE html>
<html lang="ca">
<head>
    <meta charset="UTF-8">
    <title>Carret | ShopMicro Pro</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
    <nav class="navbar"><div class="logo">EL TEU <span>CARRET</span></div><a href="index.php" class="cart-btn">Tornar</a></nav>
    <div class="container">
        <div class="card" style="width:100%; max-width:500px;">
            <?php if(isset($success)): ?>
                <h2 style="color:var(--success)">Compra realizada!</h2>
                <a href="index.php" class="btn">Seguir comprando</a>
            <?php else: ?>
                <h2>Resumen de productos</h2>
                <?php if(!empty($_SESSION['cart'])): ?>
                    <ul style="list-style:none; padding:0; text-align:left;">
                        <?php foreach($_SESSION['cart'] as $id => $qnt):
                            $p = $conn->query("SELECT * FROM products WHERE id=$id")->fetch_assoc(); ?>
                            <li style="padding:10px 0; border-bottom:1px solid #334155;">
                                <b><?php echo $p['nom']; ?></b> (x<?php echo $qnt; ?>) - <?php echo $p['preu']*$qnt; ?>€
                            </li>
                        <?php endforeach; ?>
                    </ul>
                    <form method="post"><button name="buy" class="btn" style="background:#10b981">PAGAR ARA</button></form>
                <?php else: ?><p>Buit.</p><?php endif; ?>
            <?php endif; ?>
        </div>
    </div>
</body>
</html>
```

- **logout.php:** Script de finalització de sessió. Executa session_destroy() per garantir que les credencials de l'usuari s'esborrin de la memòria volàtil del servidor web.

```
GNU nano 6.2                                logout.php
<?php
session_start();
session_unset();
session_destroy();
header("Location: login.php");
exit();
?>
```

6.3. Recursos Estàtics i Multimèdia

- **/images/**: Directori que actua com a repositori d'actius multimèdia. Conté les fotografies reals dels productes vinculades per nom a la base de dades.



- **/css/style.css**: Full d'estils que defineix la identitat visual, l'ús de variables de color per al Dark Mode i el disseny responsive de les "Cards".

```
GNU nano 6.2 style.css
/* Variables globals i paleta de colors */
:root {
  --bg: #0f172a;
  --card: #1e293b;
  --accent: #38bdf8;
  --text: #f8fafc;
  --success: #10b981;
}

/* Configuració base del document */
body {
  background: var(--bg);
  color: var(--text);
  font-family: 'Segoe UI', sans-serif;
  margin: 0;
}

/* Barra de navegació superior */
.navbar {
  display: flex;
  justify-content: space-between;
  padding: 20px 50px;
  background: #111827;
  border-bottom: 1px solid #334155;
  align-items: center;
}

/* Personalització del logotip */
.logo span {
  color: var(--accent);
  font-weight: bold;
}

/* Botó del carret d'ancoratge */
.cart-btn {
  background: var(--accent);
  color: var(--bg);
  padding: 8px 15px;
  border-radius: 6px;
  text-decoration: none;
  font-weight: bold;
}

/* Estil per a productes */
.out {
  opacity: 0.6;
}

/* Enllaços generals */
a {
  color: var(--accent);
}

/* Disposició dels productes (grid dinàmic) */
.container {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  padding: 40px;
  justify-content: center;
}

/* Targetes de producte (cards) */
.card {
  background: var(--card);
  padding: 30px;
  border-radius: 15px;
  border: 1px solid #334155;
  width: 320px;
  text-align: center;
}

/* Estils dels camps d'entrada (inputs) */
.input {
  width: 100%;
  box-sizing: border-box;
  padding: 12px;
  margin: 10px 0;
  border-radius: 8px;
  border: 1px solid #334155;
  background: #0f172a;
  color: white;
}

/* Botons d'acció principals */
.btn {
  background: var(--accent);
  color: var(--bg);
  border: none;
  padding: 14px;
  border-radius: 8px;
  cursor: pointer;
  width: 100%;
  font-weight: bold;
  margin: 10px 0;
}

/* Efectes interactius (hover) */
.btn:hover {
  background: white;
}

/* Estat deshabilitat (sense stock) */
.btn.disabled {
  opacity: 0.3;
  cursor: not-allowed;
}

/* Estils de la imatge real del producte */
.product-image {
  max-width: 100%;
  height: 200px;
  object-fit: contain;
  margin-bottom: 15px;
}

/* Format de preu */
.price {
  color: var(--accent);
  font-size: 24px;
  font-weight: bold;
  margin: 10px 0;
}

/* Indicador d'estoc disponible */
.stock {
  font-size: 0.9rem;
  color: #94a3b8;
}

/* Estructura d'informació interna de la card */
.info-row {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 20px;
}
```

Fase 2: Orquestració i Alta Disponibilitat amb Docker Swarm

7. Inicialització del clúster Swarm

Per començar amb l'orquestració, el primer que hem fet ha estat preparar l'escenari amb les tres màquines virtuals: un node Manager i dos nodes Worker.

- **Configuració del Manager:** Hem inicialitzat el clúster al node principal fent servir `sudo docker swarm init`. Això genera un token de seguretat necessari per unir la resta de nodes.
- **Unió dels Workers:** Hem copiat el token i l'hem executat al worker-1 i al worker-2 amb la comanda `docker swarm join`.
- **Verificació:** Per confirmar que la infraestructura era estable, hem executat `sudo docker node ls`.

```
alunne@swarm-manager:~/shopmicro-project$ sudo docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
yvg9twql6fohqcxhx0emv4jy *	swarm-manager	Ready	Active	Leader	29.3.0
9omh0ca61s5wmrwdygsgsz76	swarm-worker-1	Ready	Active		29.1.3
v0r7dnz032ycrd9sa1mi6k2x1	swarm-worker-2	Ready	Active		29.1.3

8. Adaptació del fitxer a Docker Stack

Hem agafat el docker-compose.yml de la Fase 1 i l'hem transformat en un docker-stack.yml per a Swarm. Per complir amb els requisits del projecte, hem afegit les següents directives de desplegament:

- **Rèpliques:** Hem configurat els microserveis d'API (product, order, user) amb un mínim de 2 rèpliques cadascun per garantir l'alta disponibilitat.
- **Constraints (Restriccions):** Hem obligat els serveis de base de dades (MySQL) a executar-se sempre al node manager (node.role == manager) per centralitzar la gestió del volum de dades.
- **Restart Policy:** Hem definit una política de reinici automàtic perquè, si un contenidor falla per qualsevol motiu, el sistema el torni a aixecar sense la meua intervenció.
- **Update Config:** Hem afegit un control d'actualització amb paral·lelisme 1, el que significa que, si faig un canvi, els contenidors s'actualitzaran d'un en un (rolling update) per no tallar el servei.

```
GNU nano 6.2          docker-stack.yml
version: '3.8'

services:
  # Infraestructura
  db-products:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: admin_password
      MYSQL_DATABASE: products_db
    networks: [backend-net]
    deploy:
      placement:
        constraints: [node.role == manager]

  db-orders:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: admin_password
      MYSQL_DATABASE: orders_db
    networks: [backend-net]
    deploy:
      placement:
        constraints: [node.role == manager]

  cache:
    image: redis:7-alpine
    networks: [backend-net]

  message-queue:
    image: rabbitmq:3-management-alpine
    networks: [backend-net]

  # Microserveis (Amb rèpliques per a l'Alta Disponibilitat)
  product-service:
    image: nginx:alpine
    networks: [backend-net]
    deploy:
      replicas: 2

  order-service:
    image: nginx:alpine
    networks: [backend-net]
    deploy:
      replicas: 2

  user-service:
    image: nginx:alpine
    networks: [backend-net]
    deploy:
      replicas: 2

  notification-service:
    image: nginx:alpine
    networks: [backend-net]
    deploy:
      replicas: 1

  # Gateway i Frontend
  api-gateway:
    image: nginx:alpine
    networks: [frontend-net, backend-net]
    deploy:
      replicas: 2

  frontend:
    image: nginx:alpine
    ports:
      - "80:80"
    networks: [frontend-net]
    deploy:
      replicas: 1

networks:
  frontend-net:
    driver: overlay
  backend-net:
    driver: overlay
```

8.1. Milliores en el fitxer Compose i Gestió de Sessions

8.1.1. Gestió de Rèpliques i Nodes (Service Scaling)

Hem configurat el product-service amb 3 rèpliques per garantir l'alta disponibilitat. A més, hem utilitzat constraints per forçar que aquests serveis web s'executin només als nodes Worker, deixant el Manager lliure per a tasques de control.

```
GNU nano 6.2          docker-compose.yml

product-service:
  image: adriarodriguez1/webserver-botiga:latest
  networks:
    - backend-net
    - frontend-net
  deploy:
    replicas: 3
    placement:
      constraints: ["node.role == worker"]
```

8.1.2. Seguretat de Dades i Punt d'Entrada (Manager Constraints)

Per protegir la integritat de la base de dades i el proxy, hem establert restriccions perquè db-products i api-gateway s'executin exclusivament al Manager. Això assegura que les dades i la porta d'accés estiguin en el node més segur del clúster.

```
api-gateway:
  image: nginx:latest
  ports:
    - "80:80"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
  networks:
    - frontend-net
    - backend-net
  deploy:
    placement:
      constraints: ["node.role == manager"]
```

8.1.3. Comunicació entre Nodes (Xarxes Overlay)

Hem implementat xarxes de tipus Overlay. A diferència de les xarxes bridge convencionals, les xarxes overlay permeten que els contenidors es comuniquin entre ells a través de diferents màquines físiques de forma xifrada i segura.

```
networks:
  frontend-net:
    driver: overlay
  backend-net:
    driver: overlay
```


8.2. Configuració del Proxy i Persistència (Nginx)

Per solucionar el problema del carret de la compra que es buidava sol, hem modificat el fitxer nginx.conf afegint la directiva ip_hash. Això fa que el balancejador recordi la IP de cada client i l'envii sempre a la mateixa rèplica, mantenint la sessió activa.

```
GNU nano 6.2 nginx.conf
events {}
http {
    upstream my_shop {
        ip_hash;
        server product-service:80;
    }

    server {
        listen 80;
        location / {
            proxy_pass http://my_shop;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

9. Desplegament de l'Stack

Un cop hem tingut el fitxer a punt, hem llançat tot l'stack amb la comanda: docker stack deploy -c docker-compose.yml shop-micro

```
alume@swarm-manager:~/shopmicro-project$ docker stack deploy -c docker-compose.yml shop-micro
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network shop-micro_backend-net
Creating network shop-micro_frontend-net
Creating service shop-micro_phpmyadmin
Creating service shop-micro_db-products
Creating service shop-micro_product-service
Creating service shop-micro_api-gateway
```

Hem verificat que tot s'hagués aixecat correctament:

- **docker stack ps shop-micro --filter "desired-state=running"**: Per comprovar com Swarm ha repartit la càrrega entre els tres nodes.

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR
p6kzfz6larz34	shop-micro_api-gateway.1	nginx:latest	swarm-manager	Running	Running 2 hours ago	
y8d2ns5qwe9t	shop-micro_db-products.1	mysql:8.0	swarm-manager	Running	Running 2 hours ago	
peym7l91tk8s	shop-micro_phpmyadmin.1	phpmyadmin:latest	swarm-manager	Running	Running 2 hours ago	
7qlu4aapddi0	shop-micro_product-service.1	adriarodriguez1/webserver-botiga:latest	swarm-worker-1	Running	Running 2 hours ago	
x9a25rhnz817	shop-micro_product-service.2	adriarodriguez1/webserver-botiga:latest	swarm-worker-2	Running	Running 42 minutes ago	
uk11td48677q	shop-micro_product-service.3	adriarodriguez1/webserver-botiga:latest	swarm-worker-1	Running	Running 2 hours ago	
ebah6wq6642k	shop-micro_product-service.4	adriarodriguez1/webserver-botiga:latest	swarm-worker-2	Running	Running 42 minutes ago	

10. Prova de tolerància a fallades

Per demostrar que el clúster realment funciona, hem simulat una caiguda crítica d'un node:

- **Aturada del Worker-2:** Hem executat `sudo systemctl stop docker` al segon worker.

```
alunne@swarm-worker-2:~$ sudo systemctl stop docker
Warning: Stopping docker.service, but it can still be activated by:
docker.socket
```

- **Verificació:** Al manager, hem vist amb `docker node ls` que el node passava a estat **Down**.

```
alunne@swarm-manager:~/shopmicro-project$ sudo docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
yvg9twql6f0hqcxhx0emv4jy *	swarm-manager	Ready	Active	Leader	29.3.0
9omh0ca61s5wmrwdygsqsgz76	swarm-worker-1	Ready	Active		29.1.3
v0r7dnz032ycrd9saini6k2x1	swarm-worker-2	Down	Active		29.1.3

- **Redistribució:** Hem comprovat amb `docker stack ps shopmicro` com l'orquestrador, en detectar la caiguda, ha mogut automàticament les rèpliques que hi havia al worker-2 cap als nodes que encara estaven vius (manager i worker-1).

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
p0kfz0lar234	shop-micro_api-gateway.1	nginx:latest	swarm-manager	Running	Running 3 hours ago		
y0dzns4qwe9t	shop-micro_db-products.1	mysql:8.0	swarm-manager	Running	Running 3 hours ago		
peym7191tk0s	shop-micro_phpmyadmin.1	phpmyadmin:latest	swarm-manager	Running	Running 2 hours ago		
7qlu4apdd0	shop-micro_product-service.1	adriarodriguez1/webserver-botiga:latest	swarm-worker-1	Running	Running 2 hours ago		
19qgs8zy332w	shop-micro_product-service.2	adriarodriguez1/webserver-botiga:latest	swarm-worker-1	Running	Running 6 minutes ago		
uk1td06077q	shop-micro_product-service.3	adriarodriguez1/webserver-botiga:latest	swarm-worker-1	Running	Running 2 hours ago		
z39sdffxsrn	shop-micro_product-service.4	adriarodriguez1/webserver-botiga:latest	swarm-worker-1	Running	Running 6 minutes ago		

- **Recuperació:** Un cop feta la prova de fallada, hem tornat a arrancar el servei amb `sudo systemctl start docker` al Worker-2. Després d'uns segons, he verificat amb `docker node ls` que el node tornava a estar en estat **Ready**. En aquest punt, el clúster torna a tenir la seva capacitat total, tot i que Swarm mantindrà les rèpliques on les ha reubicat fins que hi hagi un nou desplegament.

```
alunne@swarm-worker-2:~$ sudo systemctl start docker
```

```
alunne@swarm-manager:~/shopmicro-project$ sudo docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
yvg9twql6f0hqcxhx0emv4jy *	swarm-manager	Ready	Active	Leader	29.3.0
9omh0ca61s5wmrwdygsqsgz76	swarm-worker-1	Ready	Active		29.1.3
v0r7dnz032ycrd9saini6k2x1	swarm-worker-2	Ready	Active		29.1.3

11. Escalat en calent

Finalment, hem provat la capacitat d'escalat sense aturar l'aplicació. Hem augmentat el nombre de rèpliques del product-service a 4.

```
alunne@swarm-manager:~/shopmicro-project$ sudo docker service scale shopmicro_product-service=4
shopmicro_product-service scaled to 4
overall progress: 4 out of 4 tasks
1/4: running [=====>]
2/4: running [=====>]
3/4: running [=====>]
4/4: running [=====>]
verify: Service shopmicro_product-service converged
```

Fase 3: Seguretat al Clúster Docker Swarm

En aquesta fase ens hem centrat a assegurar la infraestructura de microserveis implementada anteriorment. Hem aplicat polítiques de gestió segura de credencials, segmentació de xarxes per limitar el radi d'atac i auditories d'imatges per detectar vulnerabilitats conegudes.

12. Anàlisi de vulnerabilitats inicial

Abans d'aplicar millores, hem analitzat el fitxer docker-stack.yml de la Fase 2. Hem identificat dos riscos principals:

- **Contrasenyes en text pla:** L'ús de MYSQL_ROOT_PASSWORD directament al fitxer és un risc, ja que qualsevol persona amb accés al repositori o al fitxer pot veure les credencials.
- **Xarxa massa oberta:** Tots els serveis compartien una visibilitat excessiva, la qual cosa facilitaria el moviment lateral d'un atacant si aconseguís comprometre el frontend.

13. Gestió de credencials amb Docker Secrets

Per solucionar l'exposició de contrasenyes, hem utilitzat **Docker Secrets**. A diferència de les variables d'entorn, els secrets s'emmagatzemen xifrats al node *Manager* i només s'entreguen en memòria als contenidors que els necessiten.

Passos realitzats:

1. Hem creat el secret al clúster:

```
alumni@swarm-manager:~/shopmicro-project$ echo "admin1234" | sudo docker secret create db_password -
g8gpcgw2rkioqykhpj664mu4s
```

2. Hem modificat el fitxer de l'stack per declarar el secret i indicar als serveis de base de dades que l'utilitzin mitjançant la variable `_FILE`.

```
alumni@swarm-manager:~/shopmicro-project$ sudo docker secret ls


| ID                        | NAME             | DRIVER | CREATED           | UPDATED           |
|---------------------------|------------------|--------|-------------------|-------------------|
| qe8wb4bcc5hsyqsk4h1gda8yb | db_name          |        | About an hour ago | About an hour ago |
| g8gpcgw2rkioqykhpj664mu4s | db_password      |        | About an hour ago | About an hour ago |
| q4v4q2aa0203p6l0mmg6vqm5w | db_root_password |        | About an hour ago | About an hour ago |


```

14. Aïllament de xarxes Overlay (Segmentació)

Per protegir el cor del nostre sistema, hem dividit el clúster en dues zones: una pública per als usuaris i una privada per a les dades.

A. Configuració de la xarxa interna (YAML)

La primera prova de seguretat és la configuració del fitxer `docker-stack.yml`. Hem definit la xarxa `backend-net` amb el paràmetre `internal: true`. Això actua com un mur: res que estigui dins d'aquesta xarxa pot sortir a Internet, ni ningú des de fora pot entrar-hi directament.

```
# Definició de xarxes
networks:
  frontend-net:
    driver: overlay
  backend-net:
    driver: overlay
    internal: true
```

B. Prova d'aïllament real (La prova del "Ping")

En lloc de mirar un codi avorrit (JSON), hem entrat dins d'un contenidor per comprovar que l'aïllament funciona.

- **La prova:** Hem intentat fer un ping a Google (8.8.8.8) des de dins de la base de dades.
- **El resultat:** El ping falla o dona error de xarxa. Això demostra que, encara que un atacant prengui el control de la base de dades, no podria enviar les nostres dades a un servidor extern perquè la xarxa no té sortida.

```
alumni@swarm-manager:~/shopmicro-project$ sudo docker exec -it 361fe05b03db sh
sh-5.1# ping -c 2 8.8.8.8
sh: ping: command not found
```

15. TLS i Certificats al clúster

Docker Swarm implementa seguretat per defecte en la comunicació entre nodes (control plane). Hem verificat que el clúster utilitza mútua TLS (mTLS).

- Cada node té assignat un certificat digital.
- L'Autoritat de Certificació (CA) interna de Swarm gestiona la rotació automàtica de certificats cada 90 dies per defecte.

```
alunne@swarm-manager:~/shopmicro-project$ sudo docker info | grep -A5 'Swarm'
Swarm: active
NodeID: yvg9twqld6fohqcxhx0emv4jy
Is Manager: true
ClusterID: i5yolanam71tpk6u86b4uuwtt
Managers: 1
Nodes: 3
```

16. Escaneig d'imatges (Imatge Segura)

Finalment, hem realitzat una auditoria de seguretat de la imatge de base de dades utilitzada mitjançant l'eina Trivy. Hem configurat l'escaneig per filtrar exclusivament les vulnerabilitats de severitat CRITICAL, que són les que requereixen una acció immediata.

```
usr/local/bin/gosu (gobinary)
=====
Total: 1 (CRITICAL: 1)
```

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
stdlib	CVE-2025-68121	CRITICAL	fixed	v1.24.6	1.24.13, 1.25.7, 1.26.0-rc.3	crypto/tls: crypto/tls: Incorrect certificate validation during TLS session resumption https://avd.aquasec.com/nvd/cve-2025-68121

```
alunne@swarm-manager:~/shopmicro-project$
```

Resultats de l'anàlisi: L'escaneig ha detectat 1 vulnerabilitat crítica activa:

- **ID de la Vulnerabilitat:** CVE-2025-68121
- **Biblioteca afectada:** stdlib
- **Versió instal·lada:** v1.24.6
- **Descripció:** Es tracta d'un error en la llibreria crypto/tls de Go que provoca una validació incorrecta dels certificats durant la represa d'una sessió TLS. Això podria permetre a un atacant suplantar connexions xifrades.

Mesures i correccions proposades:

1. **Actualització de la imatge:** Executar un nou docker pull per assegurar-nos que tenim la darrera revisió de la versió 8.0 publicada pel fabricant, ja que solen incloure pegats de seguretat de Go.
2. **Reducció de programari innecessari:** Canviar la imatge actual per una versió "Slim" de MySQL. Aquestes imatges són més lleugeres i no inclouen tantes eines auxiliars, eliminant així fitxers vulnerables que no són crítics per al funcionament de la base de dades.

Fase 4: Kubernetes — Migració i Gestió Avançada

17. Estudi i comparativa: Docker Swarm vs Kubernetes

Hem passat de Docker Swarm a Kubernetes perquè, tot i que Swarm és molt intuïtiu, Kubernetes ens ofereix una gestió molt més seriosa per a ShopMicro. En Swarm si un node queia, la recuperació era més lenta; en canvi, amb Kubernetes el sistema detecta en segons si l'aplicació està fallant i la reinicia sola. També la gestió de la xarxa interna amb Services ens permet una escalabilitat que Swarm no gestiona de forma tan robusta.

Característica	Docker Swarm	Kubernetes
Escalat	Manual (docker service scale)	Automàtic i intel·ligent
Self-healing	Reinicia el contenidor si el procés mor	Comprova la salut real de l'app (HTTP/Port)
Secrets	Només a nivell de clúster	Més segurs i integrats en el cicle del Pod
Complexitat	Baixa (Ideal per començar)	Alta (Més control però més configuració)

18. Muntatge de l'entorn amb Minikube

Per aquest desplegament hem utilitzat Minikube a la nostra màquina virtual. Hem hagut d'instal·lar les eines de kubectl per gestionar el clúster. Un cop arrencat amb el comando minikube start, hem confirmat que el clúster estava actiu.

```
alumni@swarm-manager:~/k8s-shopmicro$ kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  3h9m  v1.35.1
```

19. Creació de fitxers YAML

Aquesta és la part més important, on hem definit tota la infraestructura. Hem creat un fitxer per a cada component:

19.1. Persistència amb PVC (k8s-pvc.yaml)

El primer que he fet ha estat crear un PersistentVolumeClaim (PVC). Abans, quan reiniciava el clúster, perdia tots els productes de la base de dades. Ara, amb aquest volum d'1Gi, les dades es guarden fora del pod, així que encara que el pod de MySQL mori o es reiniciï, la info no es perd.

```
GNU nano 6.2 k8s-pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
  namespace: shopmicro
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

19.2. Base de Dades i Seguretat (k8s-db.yaml)

Per la base de dades, he fet servir MySQL 8.0. Cosetes importants aquí:

- No he posat la contrasenya al fitxer, la crido des d'un Secret (db-pass) per seguretat.
- He muntat el volum persistent just a /var/lib/mysql, que és on MySQL guarda les taules.
- He creat un servei intern que es diu db perquè la web el trobi ràpidament.

```
GNU nano 6.2 k8s-db.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: db-products
  namespace: shopmicro
spec:
  replicas: 1
  selector:
    matchLabels:
      app: db-products
  template:
    metadata:
      labels:
        app: db-products
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: db-pass
                  key: mysql-root-password
            - name: MYSQL_DATABASE
              value: "products_db"
          ports:
            - containerPort: 3306
          volumeMounts:
            - name: mysql-persistent-storage
              mountPath: /var/lib/mysql
      volumes:
        - name: mysql-persistent-storage
          persistentVolumeClaim:
            claimName: mysql-pvc
```

```
apiVersion: v1
kind: Service
metadata:
  name: db
  namespace: shopmicro
spec:
  ports:
    - port: 3306
  selector:
    app: db-products
```

19.3. Microservei de Productes (k8s-product.yaml)

Aquest és el component central de la botiga i l'hem configurat per ser totalment autònom:

- **Auto-configuració amb sed:** Com que la imatge de Docker venia amb dades de connexió que no coincidien amb el nostre clúster, hem programat un bloc command i args. Aquest executa un script de sed que sobreescriu el fitxer db.php al vol cada cop que el Pod s'engega, corregint el host i la contrasenya de la base de dades automàticament.
- **Alta Disponibilitat:** Hem definit 3 rèpliques del contenidor. Això garanteix que, si un Pod falla, els altres dos continuïn donant servei, a més de repartir la càrrega de treball entre ells.
- **Mecanismes de Self-healing:**
 - **Liveness Probe:** Monitoritza que el servidor estigui "viu". Si el contenidor es bloqueja, Kubernetes el mata i en crea un de nou automàticament.
 - **Readiness Probe:** Comprova que l'aplicació estigui llista abans d'enviar-li trànsit real, evitant errors de connexió als usuaris.
- **Exposició del servei:** S'ha configurat un servei de tipus NodePort que mapeja el port 80 del contenidor al port **30081** del clúster, permetent l'accés des del navegador de Windows mitjançant la IP de la màquina Ubuntu.

```
GNU nano 6.2 k8s-product.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: shopmicro
spec:
  replicas: 3
  selector:
    matchLabels:
      app: product-service
  template:
    metadata:
      labels:
        app: product-service
    spec:
      containers:
        - name: shop-web
          image: adriarodriguez1/webserver-botiga:latest
          command: ["/bin/sh", "-c"]
          args:
            - |
              sed -i "s/'db-products'/'db'/g" /var/www/html/db.php;
              sed -i "s/'root_password'/'products_db'/g" /var/www/html/db.php;
              apache2-foreground;
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 15
            periodSeconds: 20
```

```
apiVersion: v1
kind: Service
metadata:
  name: product-service
  namespace: shopmicro
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30081
  selector:
    app: product-service
```


19.4. Gateway d'accés (k8s-gateway.yaml)

Aquest fitxer gestiona l'entrada principal al sistema mitjançant un servidor Nginx:

- **Punt d'entrada únic:** Utilitza un servei de tipus NodePort al port 30080. Centralitza tot el trànsit extern en un sol lloc abans de moure'l cap als microserveis.
- **Proxy Invers:** Actua com a barrera de seguretat; el client no connecta directament amb la web de productes, sinó que passa primer per aquest Gateway.
- **Escalabilitat:** Està dissenyat per poder afegir més microserveis en el futur sense haver d'obrir nous ports a l'exterior, mantenint el port 30080 com a porta principal.

```
GNU nano 6.2 k8s-gateway.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-gateway
  namespace: shopmicro
spec:
  selector:
    matchLabels:
      app: api-gateway
  template:
    metadata:
      labels:
        app: api-gateway
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: api-gateway
  namespace: shopmicro
spec:
  type: NodePort
  ports:
    - port: 80
      nodePort: 30080
  selector:
    app: api-gateway
```

20. Creació del Namespace: shopmicro

Hem organitzat tots els recursos dins del namespace shopmicro. Això ens permet tenir un entorn aïllat i net, separat dels serveis per defecte de Kubernetes.

```
alumni@swarm-manager:~/k8s-shopmicro$ kubectl get ns
NAME                STATUS   AGE
default             Active   3h44m
kube-node-lease     Active   3h44m
kube-public         Active   3h44m
kube-system         Active   3h44m
shopmicro           Active   3h42m
alumni@swarm-manager:~/k8s-shopmicro$
```

21. Migració i desplegament

Hem aplicat tots els fitxers mitjançant `kubectl apply -f`. Durant el procés, hem hagut de realitzar una importació manual de la base de dades SQL al Pod de MySQL per restaurar els productes reals.

```
alumne@swarm-manager:~/k8s-shopmicro$ kubectl get pods -n shopmicro
NAME                                READY   STATUS    RESTARTS   AGE
api-gateway-76bb6fd99d-qctjv        1/1     Running   4 (73m ago)  3h37m
db-products-784ffbb5cf-clhbq        1/1     Running   3 (73m ago)  96m
product-service-6d9574696c-bl6hd    1/1     Running   4 (73m ago)  3h37m
product-service-6d9574696c-hxcvb    1/1     Running   4 (73m ago)  3h37m
product-service-6d9574696c-vwcfv    1/1     Running   4 (73m ago)  3h37m
```

```
alumne@swarm-manager:~/k8s-shopmicro$ kubectl get svc -n shopmicro
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE
api-gateway         NodePort    10.96.53.50     <none>       80:30080/TCP     3h38m
db-products          ClusterIP   10.110.146.100 <none>       3306/TCP         3h38m
product-service      ClusterIP   10.98.70.243    <none>       80/TCP           3h38m
```

```
alumne@swarm-manager:~/k8s-shopmicro$ kubectl exec -it $(kubectl get pods -n shopmicro -l app=db-products -o jsonpath='{.items[0].metadata.name}') -n shopmicro -- mysql -u root -proot_password products_db -e "source /tmp/migracion_real.sql"
mysql: [Warning] Using a password on the command line interface can be insecure.
```

22. Rolling update automàtic

Hem comprovat com Kubernetes gestiona les actualitzacions. En modificar la imatge del `product-service`, el clúster ha anat creant Pods nous i eliminant els vells de forma seqüencial, garantint que la botiga mai deixés de funcionar.

```
alumne@swarm-manager:~/k8s-shopmicro$ kubectl rollout status deployment/product-service -n shopmicro
deployment "product-service" successfully rolled out
```

23. Comprovació dels fluxos funcionals

Finalment, hem validat que la botiga funciona correctament. Mitjançant un túnel de port-forward, hem accedit a la web i hem verificat que el llistat de productes es carrega correctament des de la base de dades persistent.

```
alunne@swarm-manager:~/k8s-shopmicro$ kubectl port-forward deployment/product-service -n shopmicro 8888:80 --address 0.0.0.0
Forwarding from 0.0.0.0:8888 -> 80
Handling connection for 8888
Handling connection for 8888
```

